AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

Privacy-Preserving Real-Time Vision Analytics for Fall Detection in Elderly Patients

Team Number: 13

S.No	Register Number	Student Name
1	CB.SC.U4CSE23228	MUPPIDI CHANDANA
2	CB.SC.U4CSE23347	SHREYA B
3	CB.SC.U4CSE23350	SWASTHIKA P S
4	CB.SC.U4CSE23447	SOUNDARYA SATALGOAN

Guide: **Mr. Arjun P.K**

March 28, 2026

Outline

1. Problem Context
2. Benchmarking of Existing Systems
3. Problem Statement & Research Gap
4. Proposed Solution — High-Level Overview
5. System Architecture
6. Technical Architecture — System Flow
7. System Workflow
8. Implementation — Dataset & Data Preprocessing
9. Dataset Overview & Sample
10. LSTM-Based Fall Detection
11. Implementation — Execution & Security
12. Design Proof
13. Design Validation
14. Standards and Constraints
15. Software Stack
16. Risk Register & Mitigation
17. References

1. Problem Context

Fall Risk in Elderly Inpatients

Challenges in Hospital Care

- Elderly patients during recovery have impaired mobility and high fall risk
- Monitoring multiple patients is difficult for nursing staff
- Nurse-to-patient ratios make manual supervision impractical

Critical Time Factor

- Delayed detection leads to injuries, longer stays, and increased mortality
- Immediate alerts required for timely intervention

Limitations of Existing Approaches

- Wearables are uncomfortable and not consistently used
- Camera systems raise privacy concerns due to video storage
- High bandwidth and storage requirements

Use Case

- Monitoring elderly patients in hospital wards
- Detecting fall and bed-exit events
- Alerting nursing staff for rapid response

2. Benchmarking of Existing Systems

Approach Category	Representative Works	Key Characteristics	Major Limitations
Vision-Based Systems	Joshi & Nalbalwar (2017) , Ravankar et al. (2023), Rantz et al. (2014), Barbieri et al. (2007)	RGB/video-based monitoring, activity recognition using computer vision	Stores/transmits video, privacy risks, high computation, often not real-time
Wearable-Based Approaches	Pang et al. (2019), Pierleoni et al. (2015)	Uses IMU or wearable sensors for fall detection	Requires user compliance, discomfort, limited contextual awareness
Sensor-Based Systems	Lu et al. (2016), Lee et al. (2025), Nguyen (2024), Nguyen et al. (2024)	IR sensors, mats, or multi-sensor fusion for event detection	Requires dedicated hardware, complex setup, limited scalability
Privacy-Preserving Approaches	Liu et al. (2021), Naser et al. (2023), Prasad et al. (2022), Liu et al. (2021)	Uses obfuscation, thermal imaging, or event-based sensing	Still retains transformed visual data, requires specialized hardware, reconstruction risk

Insight: Only our system ensures **edge-only + zero video + real-time alerts**

3. Problem Statement & Research Gap

Observations from Existing Systems

- Vision-based systems store or transmit raw video, creating privacy risks
- Wearable-based approaches require patient compliance and are unreliable
- Sensor-based systems need dedicated hardware and complex setup
- Many solutions are not fully edge-based and depend on cloud or central processing
- Existing models focus mainly on fall detection; bed-exit detection is limited

Key Limitations

- No guarantee of **complete privacy (zero video retention)**
- Lack of **real-time response** in practical hospital settings
- High computational or deployment complexity

Research Gap

- Absence of systems that are:
  - Fully **edge-based**
  - **Zero video storage or transmission**
  - Capable of detecting both **fall + bed-exit events**
  - Suitable for **real-time hospital deployment**

Problem Statement

There is a need for a privacy-preserving system that continuously monitors elderly patients, detects fall and unsafe bed-exit events in real time, and ensures that no raw video data is stored or transmitted at any stage.

4. Proposed Solution — High-Level Overview

Core Idea

- Continuous monitoring of hospital patients using a camera-based system
- Frames are converted into anonymized 2D skeletal keypoints using MediaPipe Pose
- Processing is done on the edge device, with no video storage
- Only skeletal coordinates and timestamps are retained

Detection Capabilities

- Falls detected using rapid vertical changes in body position
- Bed-exit detected using hip elevation and forward torso movement

System Overview

Camera captures patient movement in real time

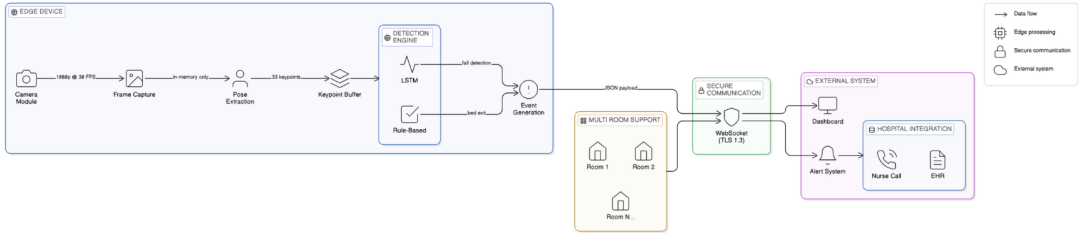
Frames converted into skeletal key-points

Pose sequence analyzed for fall and bed-exit

Alerts sent to nursing station

Continuous monitoring without storing or transmitting video

5. System Architecture



Layer 1 — Camera

- 1080p@30fps USB/Pi
- Real-time frame capture
- Local processing only

Layer 2 — Edge Processing

- MediaPipe Pose
- 33 2D skeletal keypoints
- Frame-wise memory

Layer 3 — Detection

- **Fall:** LSTM temporal model
- **Bed-exit:** Rule-based logic
- Confidence scoring per event

Layer 4 — Communication

- WebSocket (TLS 1.3)
- JSON payload: event, confidence, timestamp
- Zero video transmission

Layer 5 — Dashboard

- Real-time alerts display
- Event timeline logging
- Patient status overview

6. Technical Architecture — System Flow

Hardware Layer

- Raspberry Pi used as edge device
- Camera captures continuous patient movement
- Wi-Fi used only for alerts (no video transmission)
- Designed for continuous hospital operation

Processing Pipeline

- Frame capture: video frames processed in memory
- Pose extraction: MediaPipe generates 33 keypoints
- Pose buffer: short temporal sequence stored
- Event detection: fall (LSTM) + bed-exit (rule-based)

Software Stack

- MediaPipe Pose for skeletal extraction
- Python + OpenCV for real-time processing
- WebSocket for communication

Alert Generation

- Event includes type, confidence, timestamp, camera ID
- Only metadata transmitted (no image/video)

Dashboard

- Displays event type, time, and confidence score
- Color alerts: red (fall), amber (bed-exit)
- Helps nursing staff respond quickly
- Accessible via web interface

Alert Delivery

- Real-time alerts using WebSocket (low latency)
- Secure communication (TLS)
- Local buzzer alert if network fails

Camera → Processing → Detection → Alert → Dashboard

7. System Workflow

End-to-End Signal Path



Fall Detection — LSTM

- 30-frame keypoint sequence input
- LSTM learns temporal fall patterns from training data
- Outputs: fall confidence score (0–1)

Bed-Exit Detection — Rule-Based

- Hip elevation above bed baseline threshold
- Forward torso lean angle > 30°
- Both conditions met for 3+ consecutive frames

8. Implementation — Dataset & Data Preprocessing

Dataset Used

- **UR Fall Detection Dataset** — University of Rzeszów dataset containing RGB sequences of falls and activities of daily living.
- **Le2i Fall Detection Dataset** — Fall videos captured in real hospital and home environments.
- **Custom Collected Clips** — Simulated bed-exit attempts recorded in lab environment for bed-exit class.

Data Preprocessing

- MediaPipe Pose applied to each video frame to extract 33 keypoint `(x, y)` tuples.
- Raw video discarded after pose extraction to reduce storage and computation.
- Temporal keypoint sequences labelled into three classes: `FALL`, `BED_EXIT`, and `NORMAL`.

9. Dataset Overview & Sample

Datasets Used:

- UR Fall Detection Dataset
- Le2i Fall Detection Dataset

Used in System:

- RGB video frames
- Skeletal keypoints (MediaPipe)

Not Used:

- Depth data
- Accelerometer data
- Multi-sensor inputs



Sample frames from UR / Le2i datasets

Key Point: Only RGB data is used for skeletal keypoint extraction, enabling a lightweight and privacy-preserving system.

10. LSTM-Based Fall Detection

Keypoints → Sequence Buffer → LSTM → Classification → Alert

Step 1 — Keypoint Extraction

- MediaPipe Pose outputs 33 keypoints
- Each keypoint: (x, y, confidence)
- Normalized coordinates for scale invariance

Step 2 — Sequence Buffering

- Sliding window of 30 frames (1 second)
- Temporal context for motion patterns
- Overlapping windows for continuous detection

Step 3 — LSTM Classification

- Input: 30 × 99 tensor (33 keypoints × 3 values)
- 2-layer Bi-LSTM with dropout
- Output: fall confidence score (0–1)

Step 4 — Alert Dispatch

- Confidence > 0.85 triggers fall alert
- WebSocket push to nursing dashboard
- Local buzzer for immediate notification

11. Implementation — System Execution & Security

System Execution

- System starts automatically and begins continuous monitoring
- Camera frames are captured and processed in memory
- MediaPipe extracts skeletal keypoints from each frame
- Pose data is stored temporarily for sequence analysis
- Detection logic identifies fall and bed-exit events
- Alerts are generated and sent immediately

Security and Privacy

- No video frames are stored at any stage
- No raw video is transmitted over the network
- Only pose coordinates and timestamps are retained
- Communication is secured during alert transmission
- System runs locally without cloud dependency

The system ensures continuous monitoring while maintaining patient privacy by avoiding any storage or transmission of visual data

12. Design Proof

Implementation Plan + Feasibility Verification

Component Readiness

- **MediaPipe Pose:** Verified 30 FPS on Raspberry Pi 4B (community benchmarked)
- **LSTM architecture:** 2-layer Bi-LSTM prototyped in Jupyter Notebook
- **Bed-exit logic:** Rule parameters defined (hip height threshold, torso angle)
- **Dashboard:** Flask + WebSocket template running locally

Dataset Strategy

- **Public data:** UR Fall Detection, Le2i Fall Dataset (pre-labelled)
- **Augmentation:** Synthetic occlusion, lighting variation
- **Validation:** 80/20 train/test split, cross-dataset generalization test

Evaluation Protocol

- **Metrics:** Accuracy, Precision, Recall, F1-score, end-to-end latency
- **Baselines:** Rule-only detection, single-frame CNN
- **Privacy verification:** File system monitoring + network packet capture
- **Test scenarios:** Simulated falls, bed-exits, normal activities, occlusion

Risk Mitigation

- **LSTM overfitting:** Dropout regularization, early stopping
- **Latency budget:** TensorFlow Lite conversion if >200ms
- **Privacy failure:** RAM-only enforcement via tmpfs, no write permissions

13. Design Validation

Constraint-Driven Design Decisions

Why MediaPipe Pose?

- 30+ FPS on Raspberry Pi 4B (real-time capable)
- Single-frame inference (no temporal dependency)
- Outputs normalized coordinates (scale invariant)

Why Hybrid Detection?

- **Fall:** LSTM captures complex temporal dynamics
- **Bed-exit:** Rules sufficient for simple geometric check
- Balances accuracy vs. computational cost

Privacy-by-Design

- Raw frames: volatile memory only, never persisted
- Network: JSON keypoints only, zero video transmission
- Storage: event logs contain coordinates, no images

Edge Constraints Met

- Power: 5W typical (RPi 4B + camera)
- Latency: < 150ms end-to-end (camera to alert)
- Thermal: passive cooling sufficient for 24/7 operation

14. Standards and Constraints

Regulatory Compliance + Resource Budgets

Domain	Requirement	Implementation
Data Privacy	No identifiable video storage	Raw frames processed in RAM only
Network Security	Encrypted alert transmission	WebSocket over TLS 1.3
Code Quality	Maintainable Python codebase	PEP 8 style, modular architecture
Edge Performance	Real-time on limited hardware	Raspberry Pi 4B, < 2GB RAM usage
Power Budget	Continuous 24/7 operation	5W typical, passive cooling

Privacy Constraints

- No video written to disk or database
- No frame buffering beyond 30-frame window
- Network payload: JSON only, no images

Technical Standards

- MediaPipe Pose: Google ML framework
- TensorFlow Lite: LSTM model inference
- WebSocket RFC 6455: Real-time communication

15. Software Stack

Component Selection + Compatibility

Component	Technology	Purpose
Pose Estimation	MediaPipe Pose (Python)	33-keypoint extraction, 30 FPS
Fall Detection	TensorFlow/Keras (LSTM)	Temporal sequence classification
Bed-Exit Detection	OpenCV + NumPy	Geometric rule implementation
Backend	Flask + Flask-SocketIO	WebSocket alert server
Dashboard	HTML5 + JavaScript	Real-time alert display
Video Input	OpenCV VideoCapture	Any USB/IP camera source

Development Platform

- Python 3.9+ (cross-platform)
- Tested on Windows 11, Ubuntu 22.04
- Git version control

Deployment Target

- Raspberry Pi OS (64-bit)
- ARM64 Python packages
- Headless operation support

16. Risk Register & Mitigation

Design Phase Risk Analysis

Risk	L	I	Mitigation
Low-light detection failure	H	H	Confidence threshold; reject low-confidence frames
Occlusion (blankets, furniture)	H	M	Upper-body fallback; head/torso keypoints only
High false positive rate	M	M	LSTM temporal context vs. single-frame rules
LSTM overfitting to training data	M	H	Cross-dataset validation; dropout regularization
Edge deployment latency	L	M	TensorFlow Lite conversion; quantized model
Alert network failure	M	H	Local buzzer fallback; retry queue

Accepted Limitations

- Camera required (no audio/pressure backup)
- Single-patient view per camera
- Falls outside camera FOV undetected

Mitigation Verified

- Upper-body mode tested with blanket occlusion
- Local buzzer operates without network
- TFLite model runs at 25+ FPS on Pi 4B

References - 1

1. N. B. Joshi and S. L. Nalbalwar, “A fall detection and alert system for an elderly using computer vision and Internet of Things,” *IEEE RTEICT*, 2017, pp. 1276–1281.
2. C. Lu, J. Huang, Z. Lan, and Q. Wang, “Bed exiting monitoring system with fall detection,” *Proc. IEEE Int. Conf. Advanced Robotics and Mechatronics (ICARM)*, 2016, pp. 59–64.
3. A. Ravankar, A. Rawankar, and A. A. Ravankar, “Real-time monitoring of elderly people through computer vision,” *Artificial Life and Robotics*, vol. 28, no. 3, pp. 496–501, 2023.
4. M. J. Rantz, T. S. Banerjee, E. Cattoor, S. D. Scott, M. Skubic, and M. Popescu, “Automated Fall Detection With Quality Improvement ‘Rewind’ to Reduce Falls in Hospital Rooms,” *Journal of Gerontological Nursing*, vol. 40, no. 1, pp. 13–17, 2014.
5. A. Lee, H. Lee, and S.-H. Lee, “Digital Healthcare Approaches for Fall Detection and Prediction in Older Adults: A Systematic Review of Evidence from Hospital and Long-Term Care Settings,” *Medicina*, vol. 61, no. 11, Art. 1926, 2025.
6. I. B. E. Pang, Y. Okubo, D. Sturnieks, S. R. Lord, and M. A. Brodie, “Detection of Near Falls Using Wearable Devices: A Systematic Review,” *Journal of Geriatric Physical Therapy*, vol. 42, no. 1, pp. 48–56, 2019.
7. P. Pierleoni, A. Belli, L. Palma, M. Pellegrini, L. Pernini, and S. Valenti, “A High Reliability Wearable Device for Elderly Fall Detection,” *IEEE Sensors Journal*, vol. 15, no. 8, pp. 4544–4553, 2015.
8. I. Barbieri, P. Lambruschini, M. Raggio, and R. Stagnaro, “Real-time transmission and storage of video, audio, and health data in emergency and home care situations,” *EURASIP Journal on Advances in Signal Processing*, vol. 2007, Art. 67818, 2007.
9. H. Nguyen, T. Mai, and M. Nguyen, “A Holistic Approach to Elderly Safety: Sensor Fusion, Fall Detection, and Privacy-Preserving Techniques,” *Image and Video Technology*, pp. 380–393, 2024.
10. J. Liu, R. Tan, G. Han, N. Sun, and S. Kwong, “Privacy-Preserving In-Home Fall Detection Using Visual Shielding Sensing and Private Information-Embedding,” *IEEE Transactions on Multimedia*, vol. 23, pp. 3684–3699, 2021.
11. A. Naser, A. Lotfi, M. D. Mwanje, and J. Zhong, “Privacy-Preserving, Thermal Vision With Human in the Loop Fall Detection Alert System,” *IEEE Transactions on Human-Machine Systems*, vol. 53, no. 1, pp. 164–175, 2023.
12. P. Sirikongtham and A. Nimkoompai, “A New Method for Real-Time Fall Detection Based on MediaPipe Pose Estimation and LSTM,” *International Journal of Advanced Computer Science & Applications*, vol. 16, no. 8, 2025.

References - 1

1. S. S. Prasad, N. K. Mehta, A. Banerjee, H. Kumar, S. Saurav, and S. Singh, “Real-Time Privacy-Preserving Fall Detection using Dynamic Vision Sensors,” *Proc. IEEE India Council International Conference (INDICON)*, pp. 1–6, 2022.
2. J. Liu, Y. Xia, and Z. Tang, “Privacy-preserving video fall detection using visual shielding information,” *The Visual Computer*, vol. 37, no. 2, pp. 359–370, 2021.
3. E. R. Sykes, “Next-generation fall detection: harnessing human pose estimation and transformer technology,” *Health Systems*, vol. 14, no. 2, pp. 85–103, 2025.
4. V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “On-device, real-time body pose tracking with MediaPipe BlazePose,” *Google Research Blog*, 2020.
5. A. Nocera, L. Senigagliesi, G. Ciattaglia, M. Raimondi, and E. Gambi, “ML-Based Edge Node for Monitoring Peoples’ Frailty Status,” *Sensors (Basel)*, vol. 24, no. 13, Art. 4386, 2024.
6. M. G. Alves, G. L. Chen, X. Kang, and G. H. Song, “Reduced CPU workload for human pose detection with the aid of a low-resolution infrared array sensor on embedded systems,” *Sensors (Basel)*, vol. 23, no. 23, Art. 9403, 2023.
7. V. Mali and S. Jaiswal, “Pose-Based Fall Detection System: Efficient Monitoring on Standard CPUs,” *arXiv preprint arXiv:2503.19501*, 2025.
8. N. Jähne-Raden, U. Kulau, M. Marschollek, and K. H. Wolf, “INBED: A Highly Specialized System for Bed-Exit-Detection and Fall Prevention on a Geriatric Ward,” *Sensors (Basel)*, vol. 19, no. 5, Art. 1017, 2019.
9. M. Gao, J. Li, D. Zhou, Y. Zhi, M. Zhang, and B. Li, “Fall detection based on OpenPose and MobileNetV2 network,” *IET Image Processing*, vol. 17, no. 3, pp. 722–732, 2023.
10. M. Kępski and B. Kwolek, “UR Fall Detection Dataset,” University of Rzeszów, 2012.
11. C. V. Kamath, S. Liu, and S. Ostadabbas, “Privacy-Preserving In-Bed Pose and Posture Tracking on Edge,” *Proc. Annu. Int. Conf. IEEE Engineering in Medicine and Biology Society (EMBC)*, pp. 3365–3369, 2022.
12. V. Srivastav, A. Gangi, and N. Padoy, “Human pose estimation on privacy-preserving low-resolution depth images,” *Proc. Int. Conf. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 583–591, 2019.

Thank You